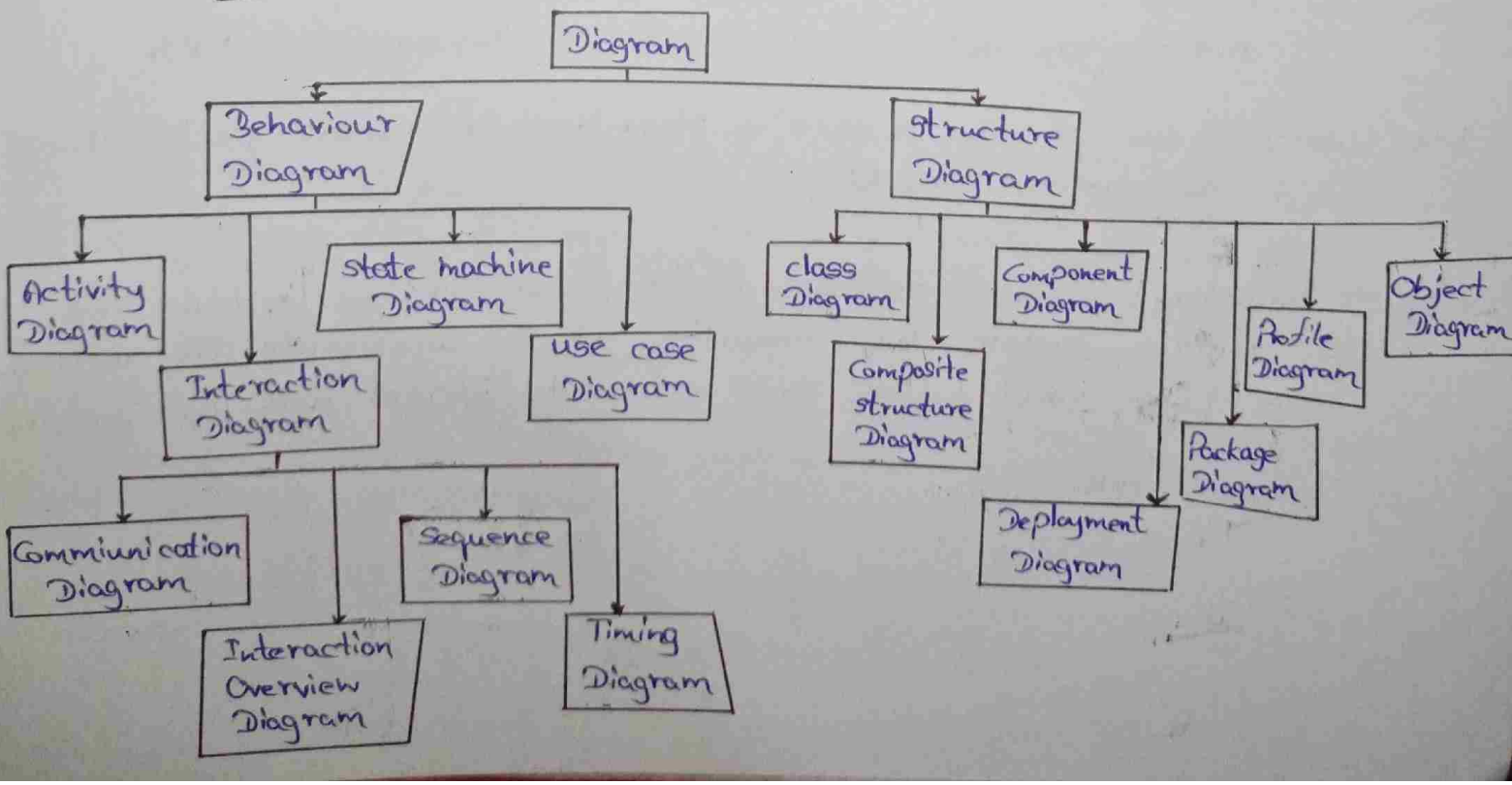
Unified Modeling Language (UML)

* UML - Unified Modeling Language is a general-purpose modeling language.
* The main aim of UML is, "define a standard way to visualize the way a system has been designed."
* This is quite similar to blueprints. UML is not a programming language. It is a 'Modeling Language'.
* We use UML diagrams to show the behavior and structure of a system.

Advantages of UML

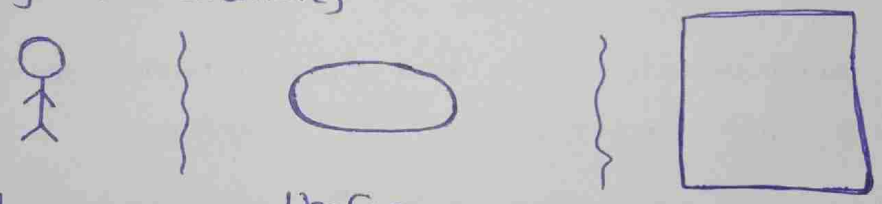
1. Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate among them.
2. Essential to communicate with non-programmers about requirement functionalities and processes of the system.
3. Time is saved when we can visualize processes, user interactions and the static structure of the system.

Types of UML diagrams



Use Case Diagrams

* This is a visual representation that represent the interactions between users and a system.
* This captures functional requirements of a system, showing how different users engage with various use cases within the system.
* Advantages of 'use case diagrams' are;
  - ⊛ provide high-level overview of a system's behaviour
  - ⊛ useful for stakeholders, developers and analysts to understand how a system is intended to operate from the user's perspective.
  - ⊛ Defining system scope and requirements.
* Use case diagram notation;



Actors :- External entities that interact with the system. This can include users, other systems or hardware devices. Actors are initiate use cases and recieve outcomes.

Use Cases :- These are the scenes in play. These represent specific things the system can do.

System Boundary :- visually represents the scope or the limit of the system. This indicating which components are internal and which are externally interact with the system.

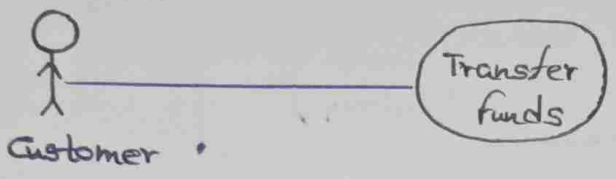
'Relationships' of the use case diagrams represents the interactions between the components of a system.

There are many types of relationships;

i) Association Relationship

Represents the interaction between an actor and a use case. Denoted by a line connecting the actor to the use case.

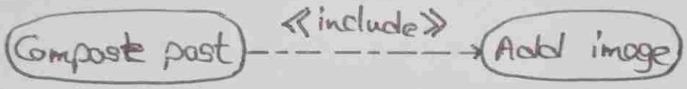
E.x. Online banking system



ii) Include Relationship

Indicates that a use case includes the functionality of another use case. Denoted by a dashed arrow pointing from the including use case to the included use case.

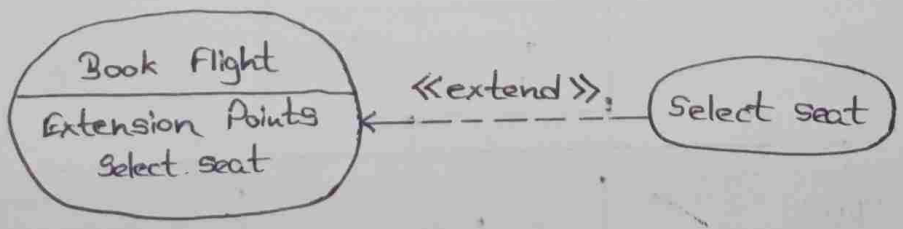
E.x. Social Media Posting



iii) Extend Relationship

Illustrates that a use case can be extended by another use case under specific conditions. Denoted by a dashed arrow.

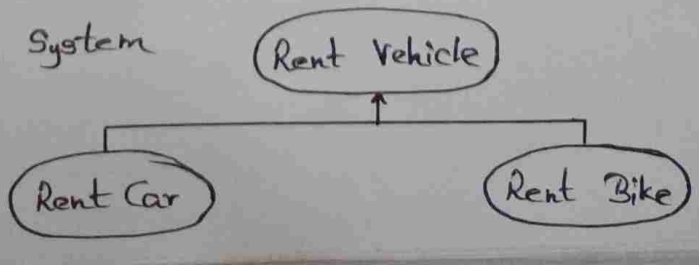
E.x. Flight Booking System



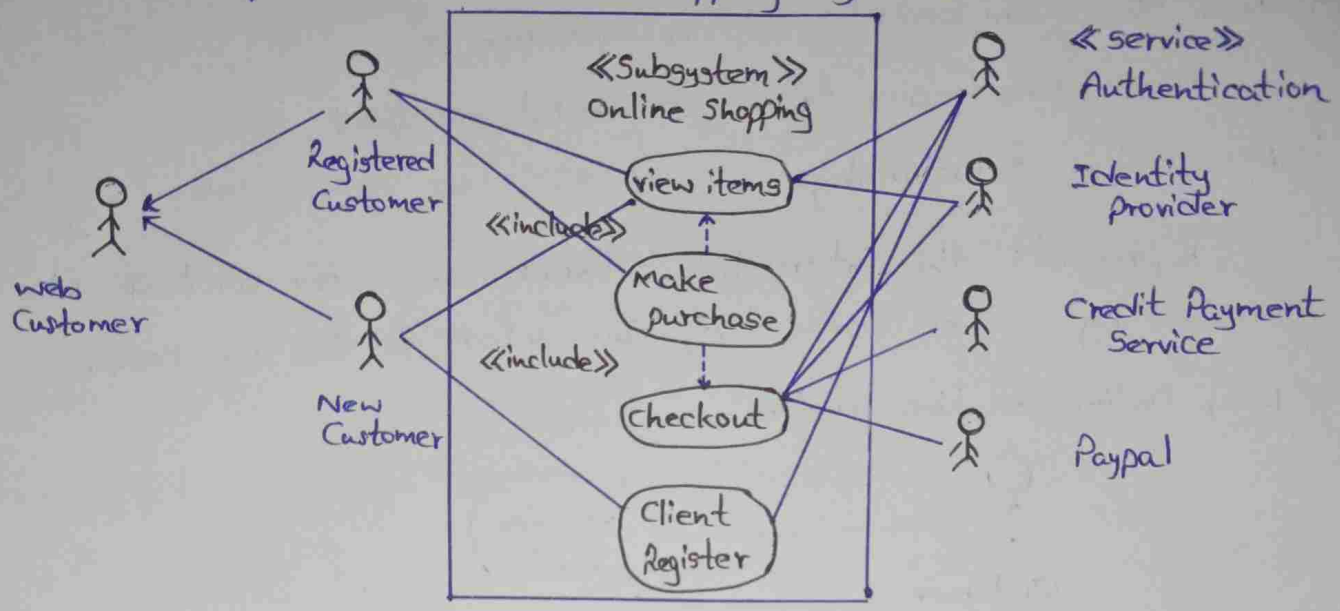
iv) Generalization Relationship

Indicating that one use case is a specialized version of another. Denoted by arrow pointing from the specialized use case to the general use case.

E.x. Vehical Rental System



* This is a example of an online shopping system.

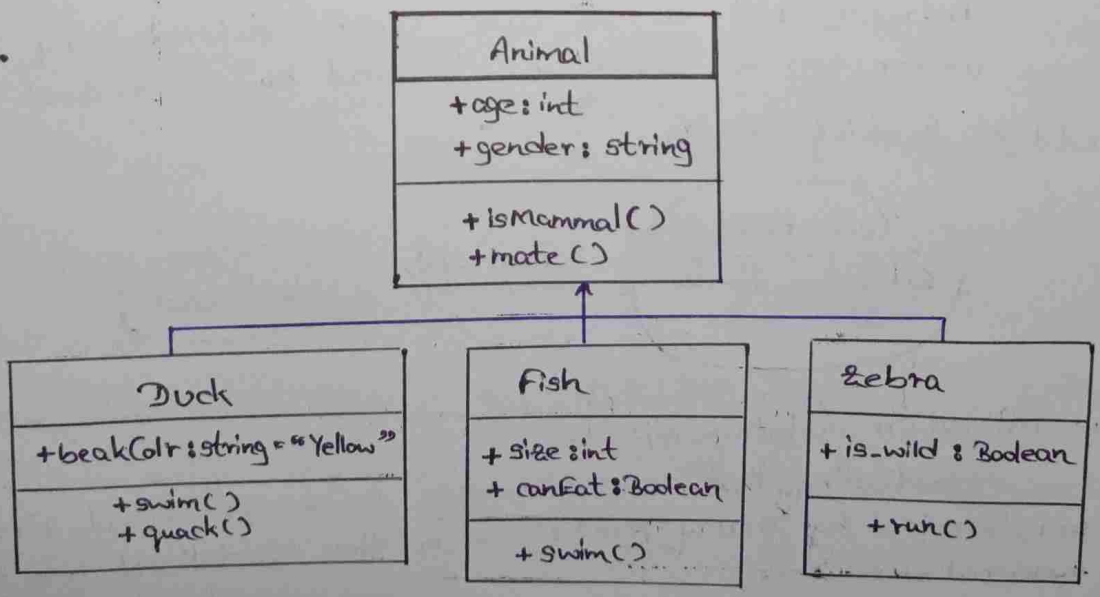


Class Diagrams

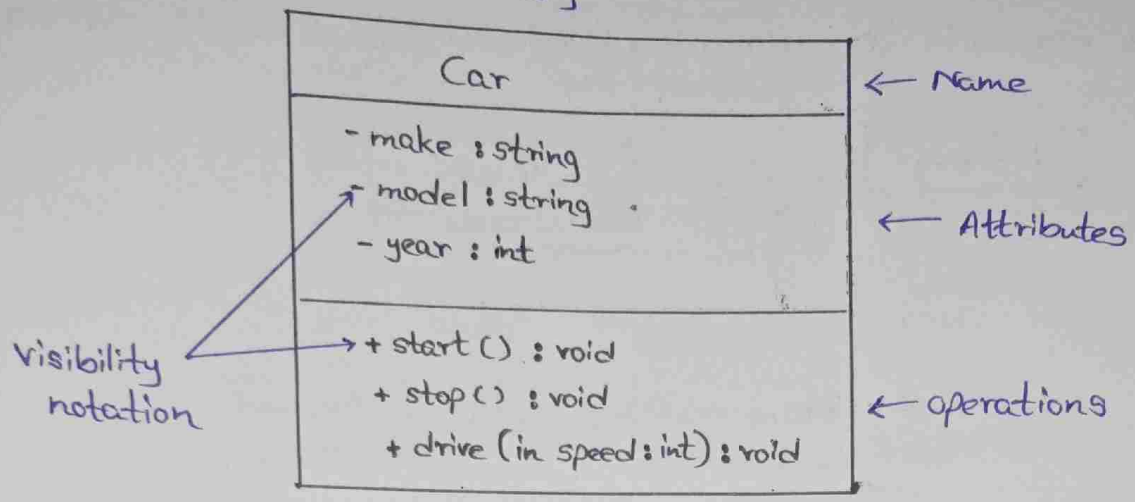
This is a diagram that represent its classes, attributes, methods and relationships between them. It helps to developers and designers to understand how the system is organized and how its components interact.

In these diagrams, classes are represented by using boxes, each box containing three compartments for the class name, attributes and methods. Lines connecting classes illustrate associations, showing relationships such as one-to-one or one-to-many.

E.x.



class notation is as follows;

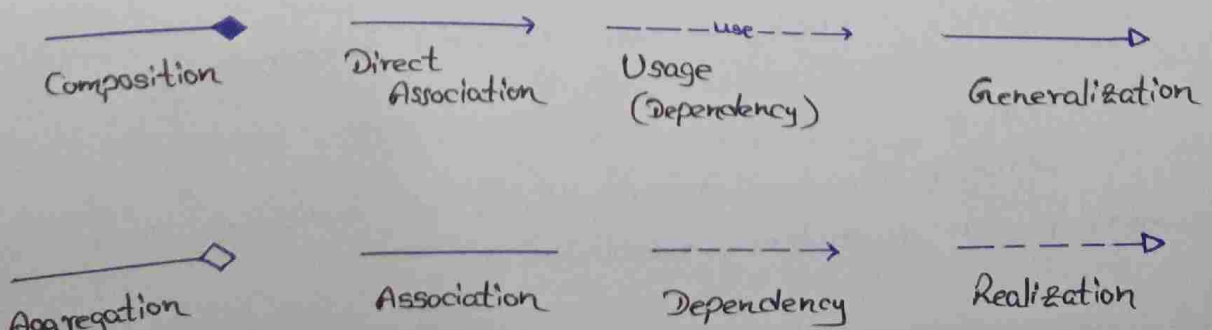


* 'Visibility notation' indicates the access level of attributes and methods.
  - (a) + → for public (visible to all classes)
  - (b) − → for private (visible only within the class)
  - (c) # → for protected (visible to subclasses)
  - (d) ~ → for package or default visibility (visible to classes in the same package)

* "Parameter Directionality" is the "indication of the flow of information between classes through method parameters. This specifies the parameter whether an input, output or both.

* There are three main parameter directionality notations;
  i) In (input)
  ii) Out (output)
  iii) InOut (input and output)

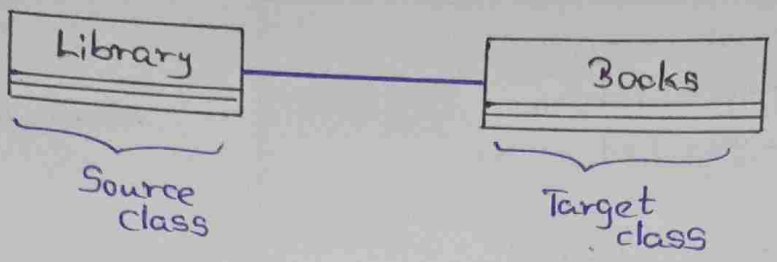
* There are several types of relationships in object-oriented modeling, each serving a specific purpose. Such as;



1.) Association :-

This represents a bi-directional relationship between classes.

E.g.

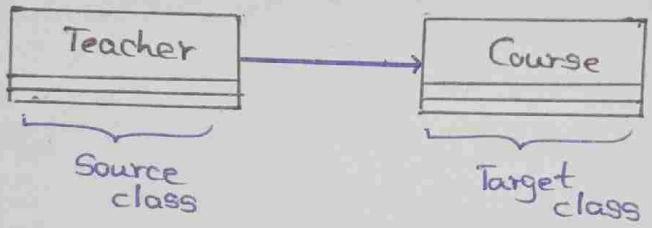


* Each library contains multiple books, and each book belongs to a specific library.

2.) Direct Association :-

Represents a relationship between two classes where the association has a direction, indicating that one class is associated with another in a specific way.

E.g.

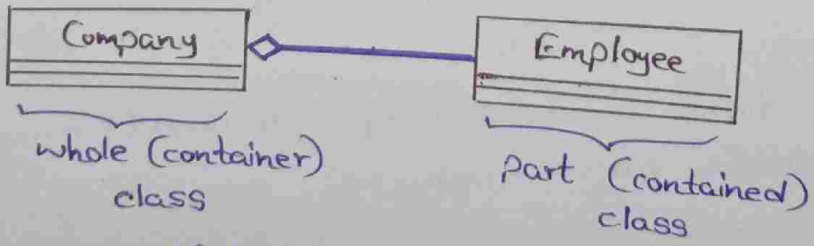


* A "Teacher" class is associated with a "Course" class in a university system.

3.) Aggregation :-

Special form of association that represents a "whole-part" relationship. This denotes a stronger relationship where one class contains or is composed of another class.

E.g.

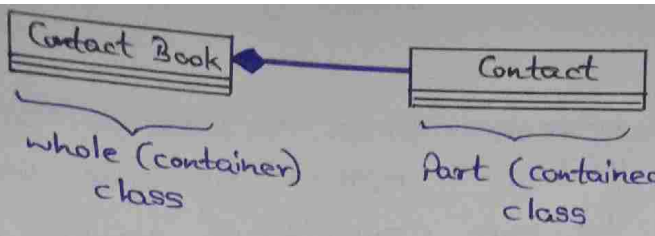


* if the company ceases to exist, the employees can still exist independently.

4.) Composition :-

Stronger form of 'aggregation'. This indicating more significant ownership or dependency relationship.

E.x.

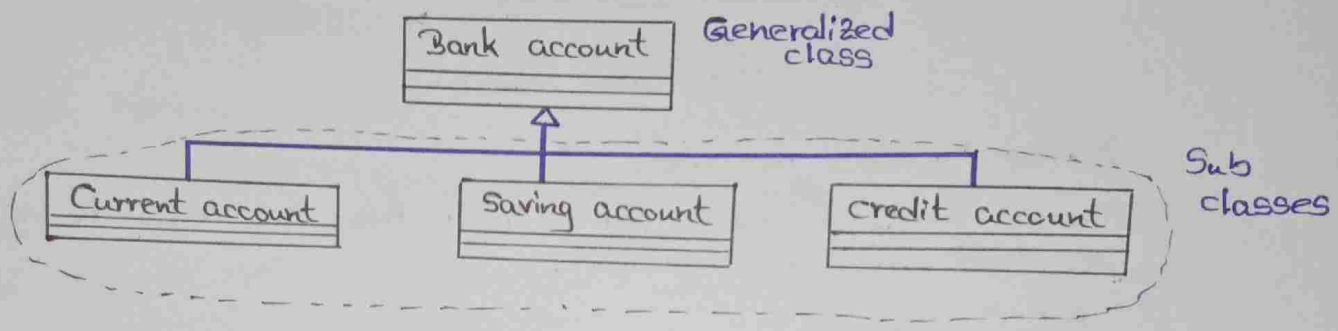


* if the contact book is deleted or destroyed, all associated contact entries are also removed.

5.) Generalization (Inheritance) :-

One class inherits the properties and behaviors of another class.

E.x.



* Advantages of class diagrams;

i.) Providing a clear view of its architecture.
ii.) Serve as a visual tool for communication among team members.
iii.) Guide developers in coding by illustrating the design.

Sequence diagrams

These diagrams used to visualize the interaction between objects in a sequencial order. Essential tool for modeling dynamic behavior in a system.

* Sequence diagram notations are as follows;

(i.) Actors :-

A role where interacts with the system and its objects.

(ii.) Lifelines :-

A named element which depicts an individual participant in a sequence diagram. Basically each instance in a sequence diagram is represented by a *lifeline*. Lifeline elements located at the top in a sequence diagram.

(iii.) Messages :-

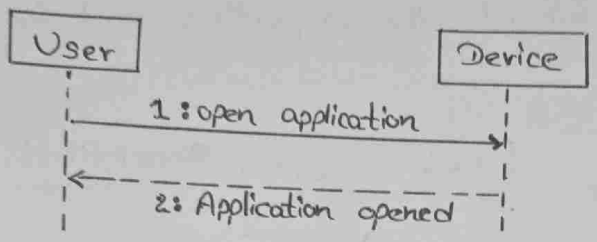
Use for communicate between objects. The message appear in a sequential order on the lifeline. We represent messages using arrows. Lifelines and Messages are the core elements of sequence diagrams.

There are two main types of messages;

i) Synchronous messages

waits for reply before the interaction can move forward. we use a solid arrow to represent a synchronous messages.

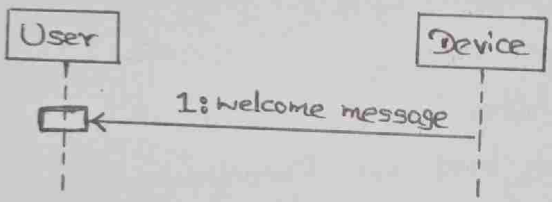
E.x.



ii) Asynchronous messages

Does not wait for a reply from the receiver. we use a solid arrow to represent.

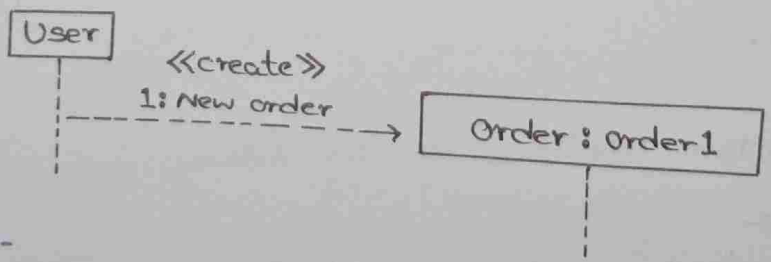
E.x.



(iv.) Create messages :-

Use to instantiate a new object in the sequence diagram. This is represented by a dotted arrow and also "create" word labelled on it.

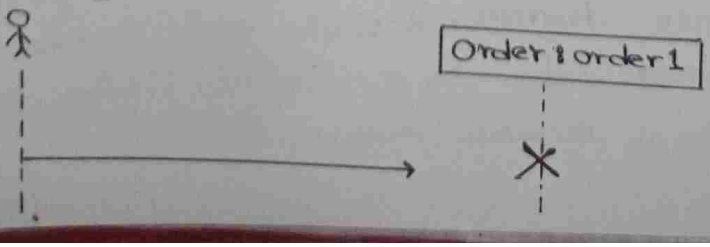
E.x.



(v.) Delete message :-

we use delete message to delete an object, when an object is deallocated memory or destroyed within the system. Represented by an arrow terminating with a cross.(X)

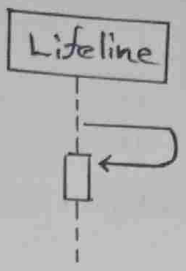
E.x.



(vi.) Self message :-

Represented by 'U shaped' arrow.

E.x.



(vii.) Reply message :-

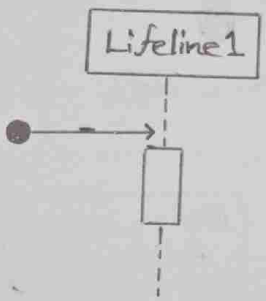
Used to show the message being sent from the receiver to the sender. Represent by open arrow head with a dotted line.

E.x.

(viii.) Found message :-

Represent a scenario where an unknown source sends the message. Denote by an arrow directed towards a lifeline from an end point.

E.x.



(ix.) Lost message :-

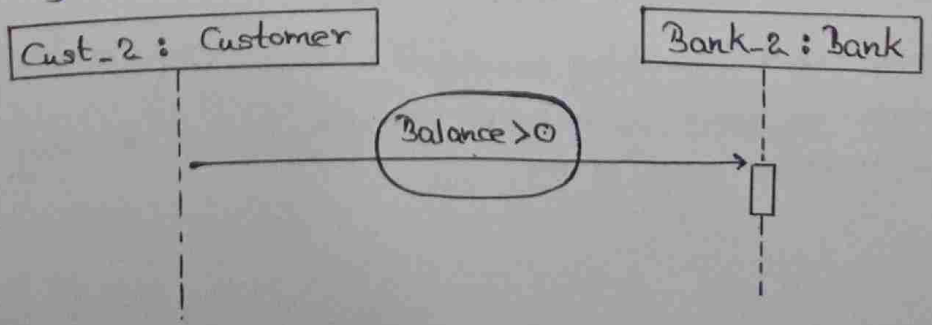
Represent a scenario where the recipient is not known to the system. Denote by an arrow directed towards an end point from a lifeline.

E.x.

(x.) Guards :-

Use to restrict the flow of messages on the pretext of a condition being met.

E.x.

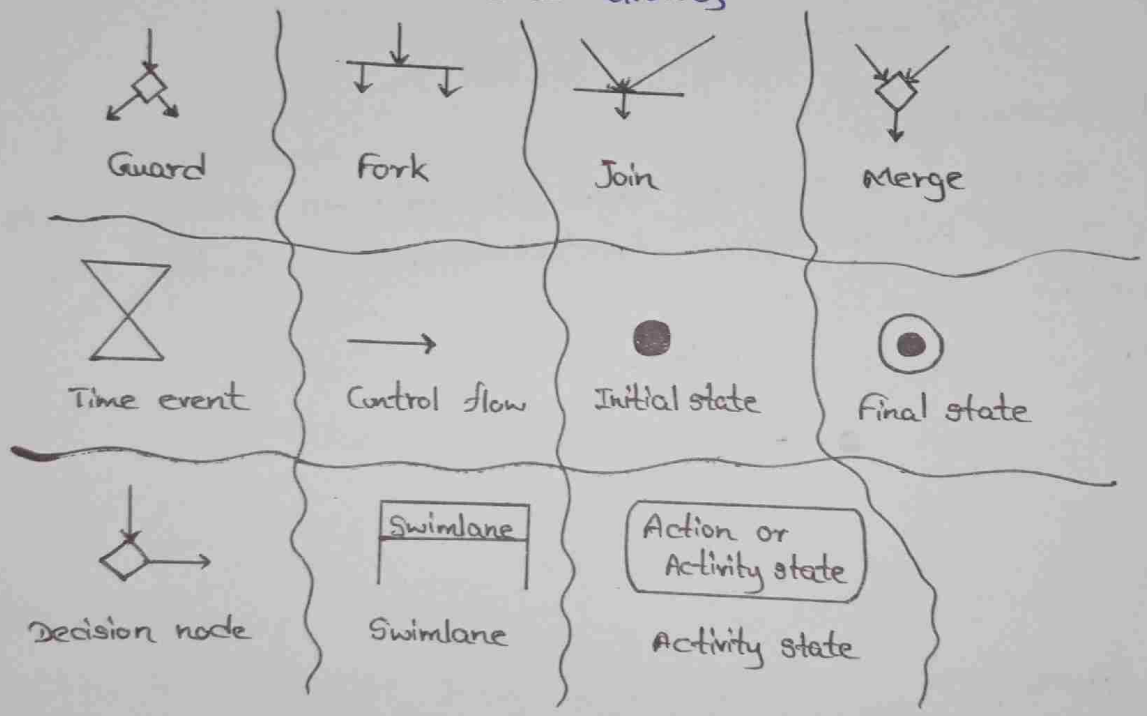


Activity diagrams

These help to visualize workflows, processes or activities within a system.

* Advantages (Usage) of Activity diagrams are;
  - Helps to understand the dynamic behavior of a system.
  - Simply clarifying complex logic.
  - Gives a clear picture of the system design.
  - Describing use cases clearly.

* Activity diagram notations are as follows;



(i.) Initial state :-

The starting state before an activity takes place.

(ii.) Action or Activity state :-

Represents execution of an action on objects or by objects.

(iii.) Action Flow or Control Flows :-

Refers to as paths and edges. Shows the transition from one activity state to another activity state.

(iv.) Decision Node and Branching :-

when we need to make a decision before deciding the flow of control, we use decision node.

(v.) Guard :-

Shifts the flow along a particular direction, if a condition is acceptable or not.

(vi.) Fork :-

Use to support concurrent activities.

(vii.) Join :-

Use to support concurrent activities converging into one.

(viii.) Merge or Merge Event :-

Use to combine two or more activities into one, if the control proceeds onto the next activity irrespective of the path chosen.

(ix.) Swimlanes :-

Use to grouping related activities in one column.

(x.) Time Event :-

Refers to an event that stops the flow for a time.

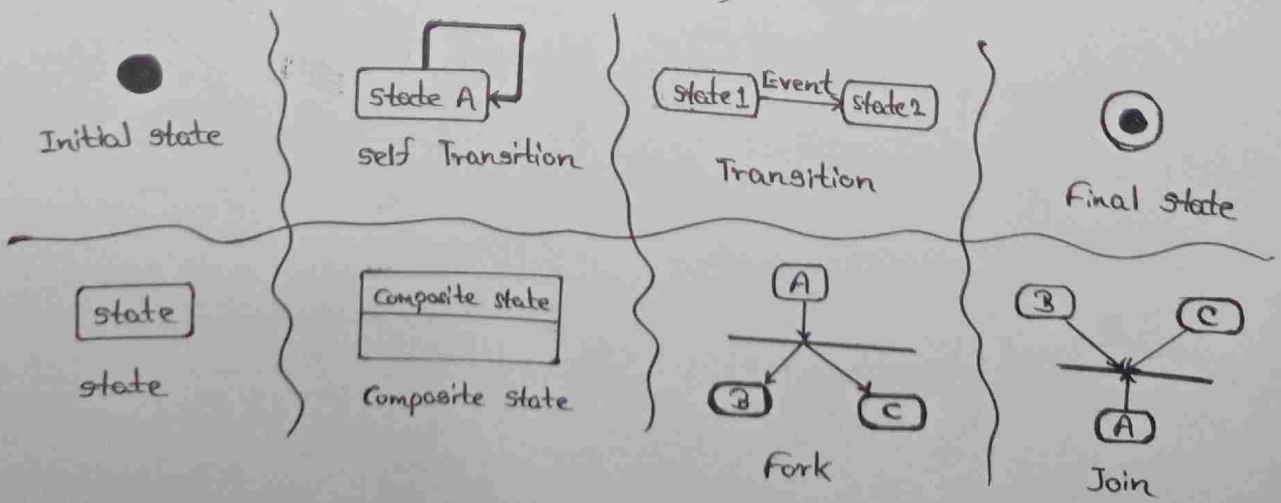
(xi.) Final State or End State :-

Represent the end of an activity or a process.

State Machine Diagrams

These are use to represent the condition of the system or part of the system at finite instances of time. Also known as; "State Diagrams" or "state chart Diagrams".

A state Machine Diagram notations as follows;



(i.) Initial state :-

starting state of a system or a class.

(ii.) Transition :-

Use a 'solid arrow' to represent the transition.

(iii.) State :-

Represent the conditions or circumstances of an object of a class.

(iv.) Fork :-

Represent a state splitting into two or more concurrent states.

(v.) Join :-

Represent two or more states concurrently converge into one on the occurrence of an event.

(vi.) Self Transition :-

Use when the state of the object does not change upon the occurrence of an event.

(vii.) Composite state :-

Represent a state with internal activities.

(viii.) Final State :-

End state of a system or a class.